Tecnológico de Monterrey

Evidence 2: Review 2

María Guadalupe Soto Acosta A00228158

Jimena Díaz Franco A01403295

Ilan Gómez Guerrero A01621122

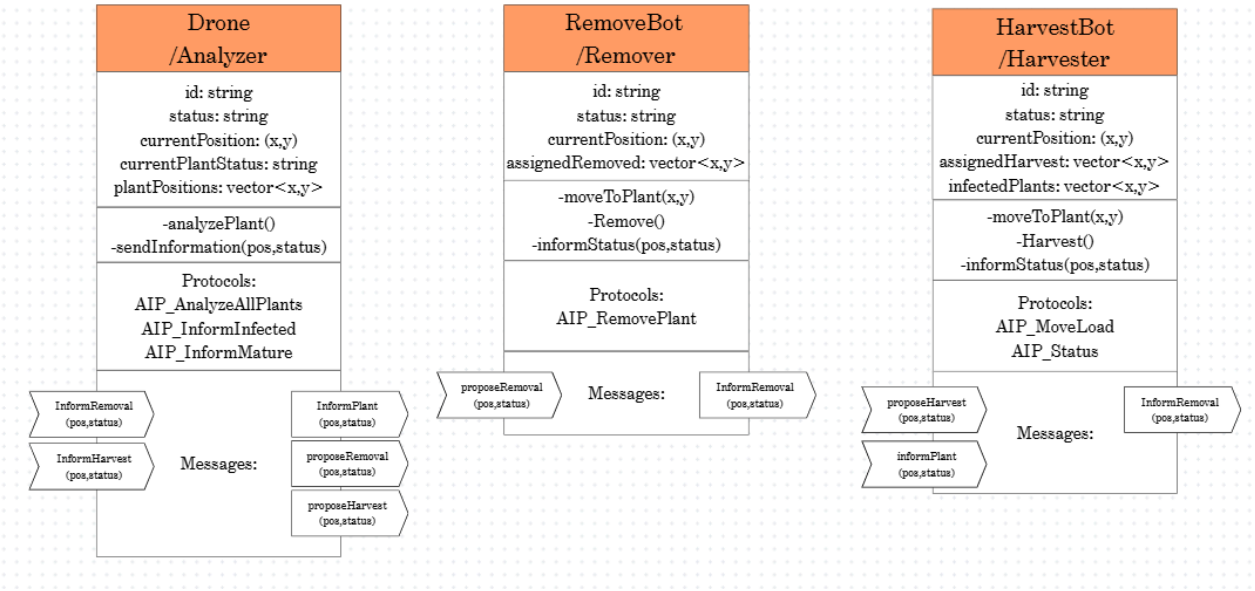
Francisco Raziel Andalón Aguayo A01640235

November 18th, 2025

Modeling of Multi-Agent Systems with Computer Graphics

Group 301

Agent Class Diagram



The agent diagram illustrates the organization of the multi-agent system responsible for monitoring and processing plants in the simulated environment. The system consists of three specialized agents: Drone/Analyzer, RemoveBot, and HarvestBot.

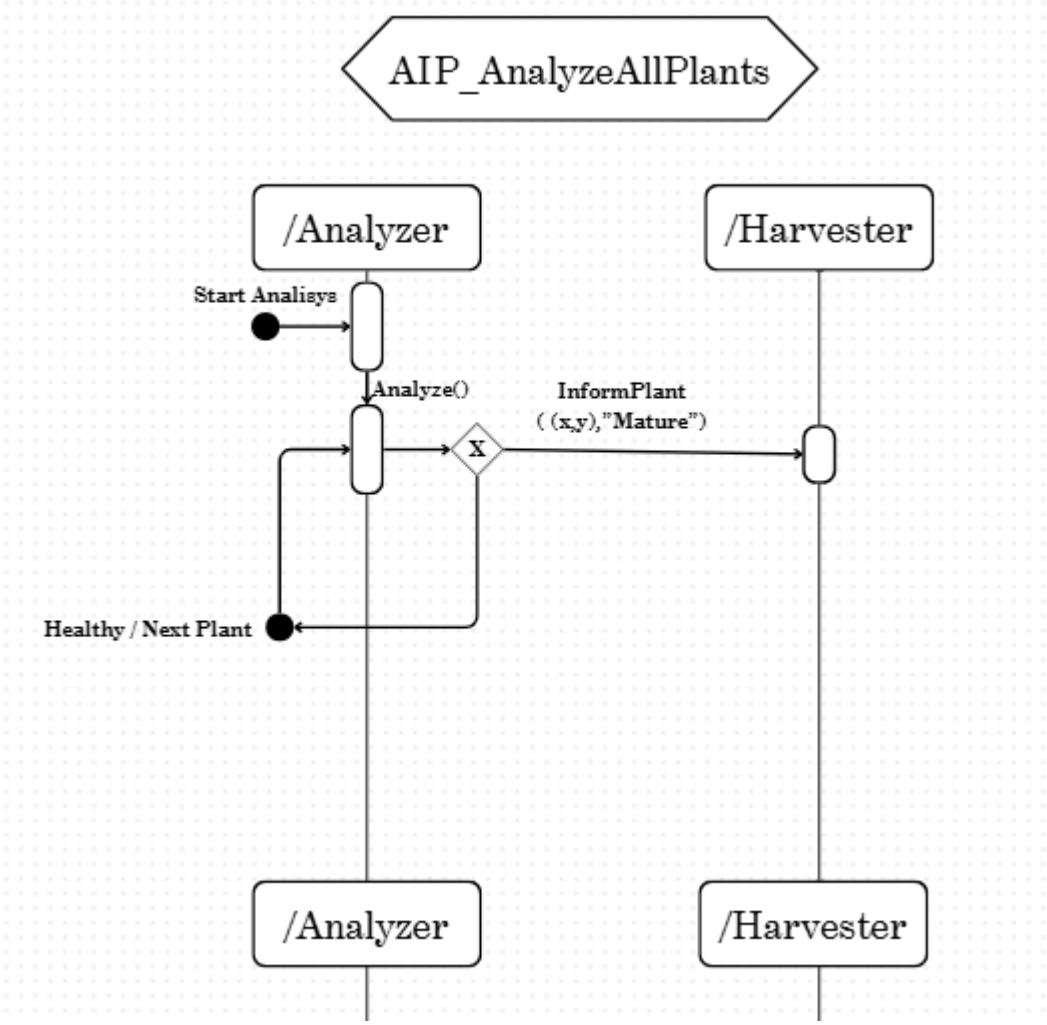
The Analyzer is responsible for perceiving the environment and detecting infected or mature ripe plants. It processes images, and generates a position list. It uses protocols such as *AIP_AnalyzeAllPlants*, *AIP_InformInfected* and *AIP_InformHarvesting* to communicate the results to the corresponding agents.

The RemoveBot receives the infected plants positions through the messages *InformRemoval* or *proposeRemoval*. Its behavior focuses on navigating to the indicated locations, removing infected plants, and reporting the status using *informStatus*.

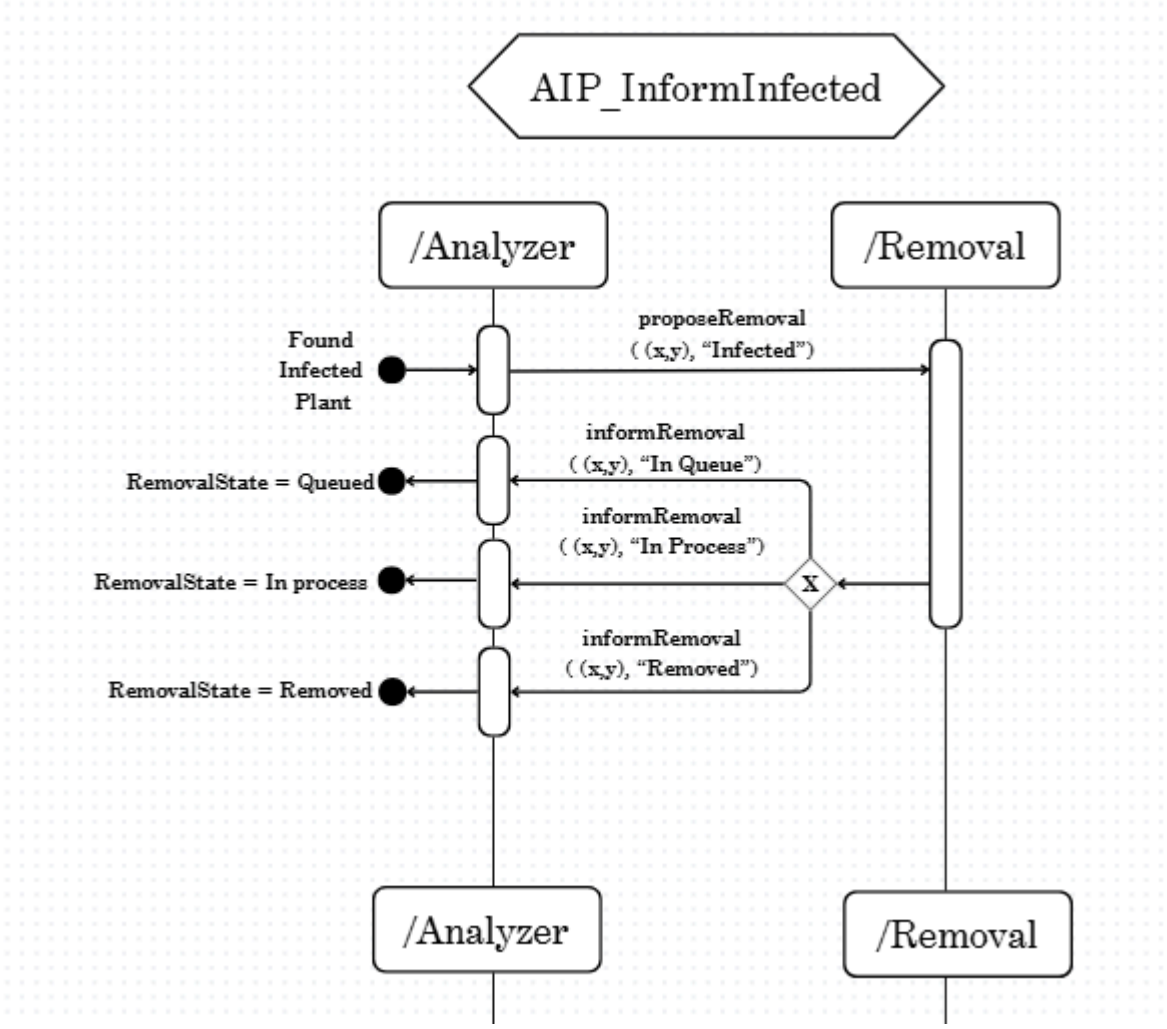
The HarvestBot coordinates with the Analyzer through the *InformHarvest* message and other messages related to the harvest proposal. Its role is to move to the locations of ripe plants and perform the corresponding harvest.

Each agent contains attributes that represent its internal knowledge and methods that represent its capabilities. In addition, the diagram includes the protocols and messages necessary for communication between agents, allowing for the modeling of cooperation and dynamic task assignment within the system.

Interaction Diagram (AIP)



This diagram describes the interaction protocol in which the Analyzer agent scans each plant in the environment, determines its status, and coordinates with the Harvester if necessary.
The protocol begins with the Analyzer initiating a new analysis cycle. The agent captures the plant's data, processes it, and evaluates whether the plant is mature. A decision point determines the outcome: if the plant is mature, the Analyzer sends an *InformPlant({x, y}, "Mature")* message to the Harvester agent. The Harvester receives the message and queues the location as a new harvest task. If the plant is not mature, the Analyzer simply continues with the next plant in the dataset.
Once the analysis for the current plant is finished, the Analyzer proceeds to the next plant in the environment. This protocol repeats for all plants until the system stops.



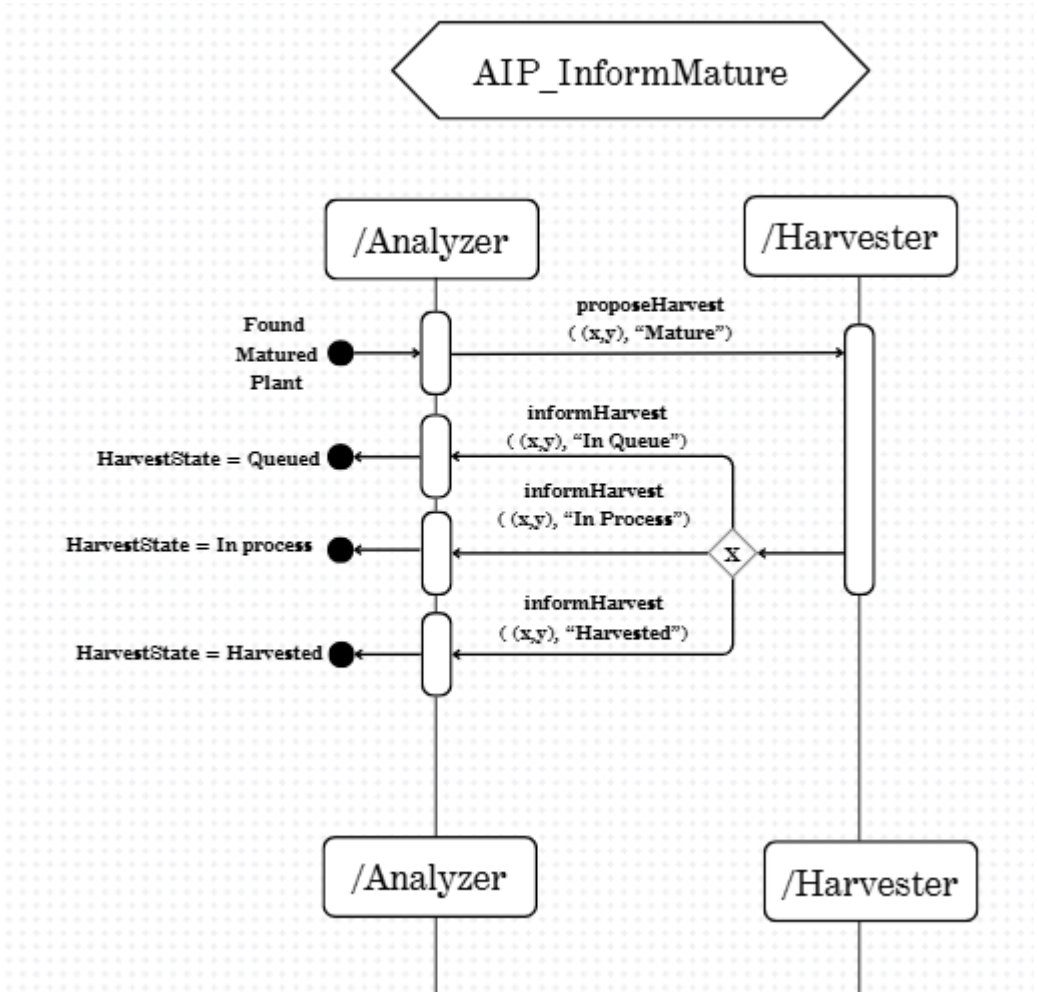
AIP_InformInfected details the protocol used when the Analyzer detects that a plant is infected. This AIP regulates how information flows between the Analyzer and the Remover Agent, enabling dynamic task assignment and status reporting.

The protocol begins when the Analyzer identifies an infected plant. It then sends a *proposeRemoval({x,y}, "Infected")* message to the Remover, indicating the location that requires removal.

Once the Remover receives the proposal, it may be in different internal states depending on its current workload:

- **Queued:** the plant has been added to the removal queue but not yet processed. The Remover responds with *informRemoval({x,y}, "In Queue"*).
- **In Process:** The Remover is actively navigating to or removing the infected plant. It responds with *informRemoval({x,y}, "In Process")*.
- **Removed:** The infected plant has already been processed. It responds with *informRemoval({x,y}, "Removed").*

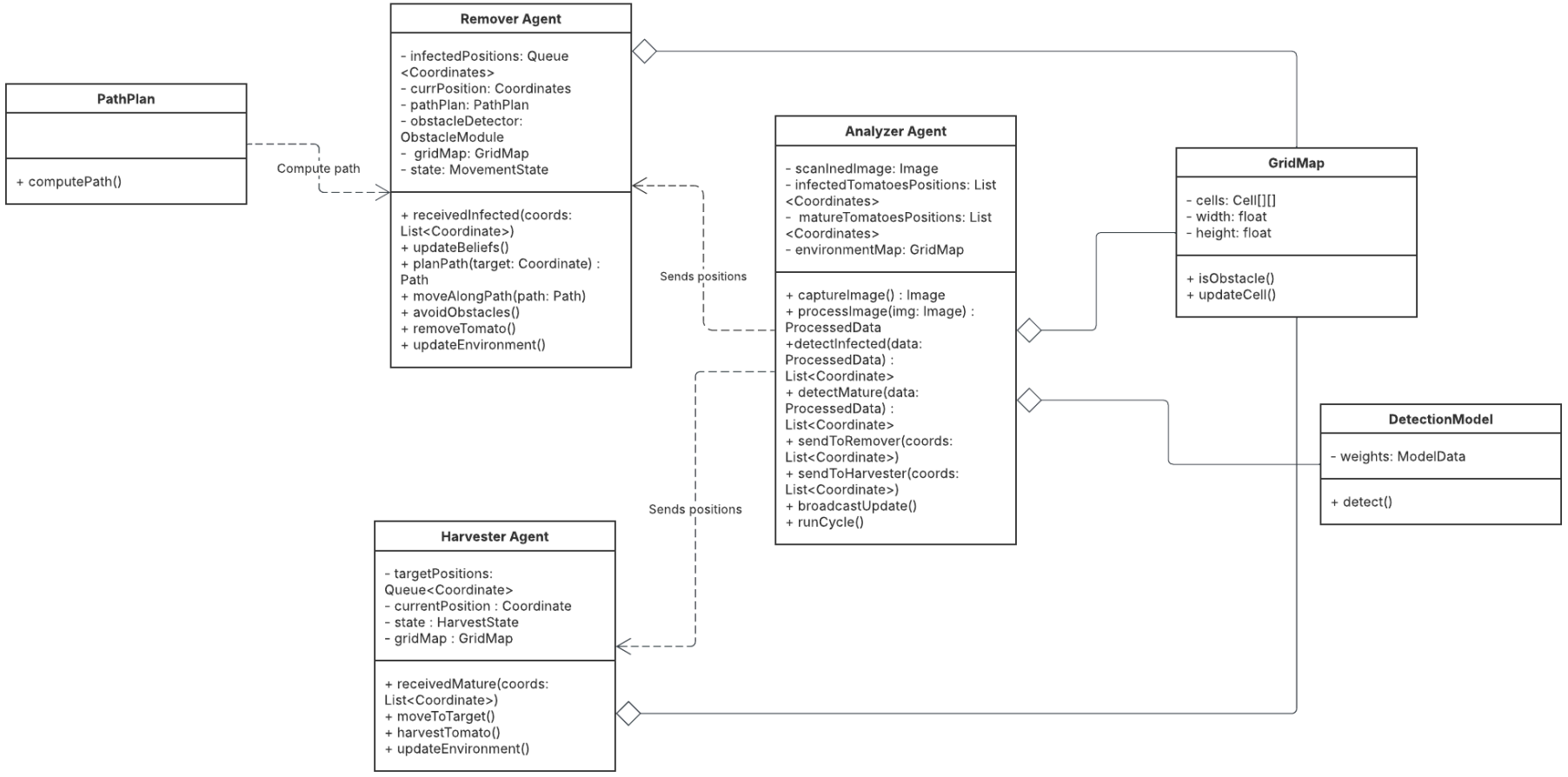
The protocol ensures that the Analyzer always stays informed about the progress of the removal tasks.



AIP_InformMature defines how the Analyzer informs the Harvester when a mature plant is detected and how the Harvester processes that information.

When the Analyzer determines that a plant is mature, it sends an *InformHarvest({x,y}, "Mature")* message to the Harvester. The Harvester agent adds the position to its internal queue, schedules the harvesting task, and later reports back with a message such as *({x,y}, "Harvested")* depending on the status of the process as the AIP_InformInfected protocol.

Standard Class Diagram



The class diagram models the main classes (AnalyzerAgent, RemoverAgent, and HarvesterAgent) along with their attributes, methods, and relationships with auxiliary modules such as GridMap, DetectionModel and PathPlan.

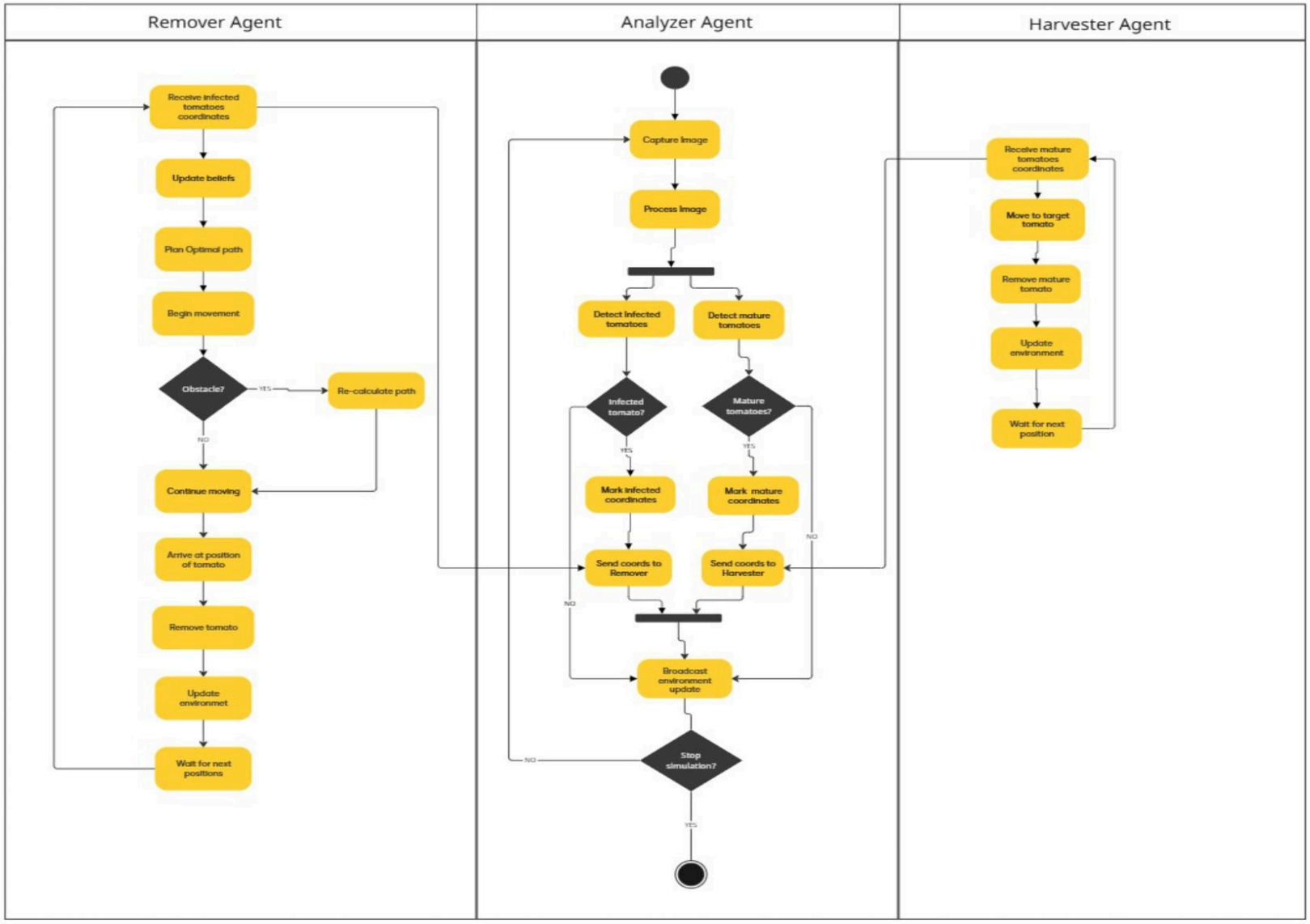
In the AnalyzerAgent, its attributes include detection models for infected and ripe tomatoes, as well as lists of detected positions and a map of the environment. It maintains aggregation relationships with GridMap, which represents the distribution of the environment, and with DetectionModel, which encapsulates the detection logic. In addition, it defines methods such as captureImage(), processImage(), and sendToRemover() or sendToHarvester(), which facilitate communication with the other agents.

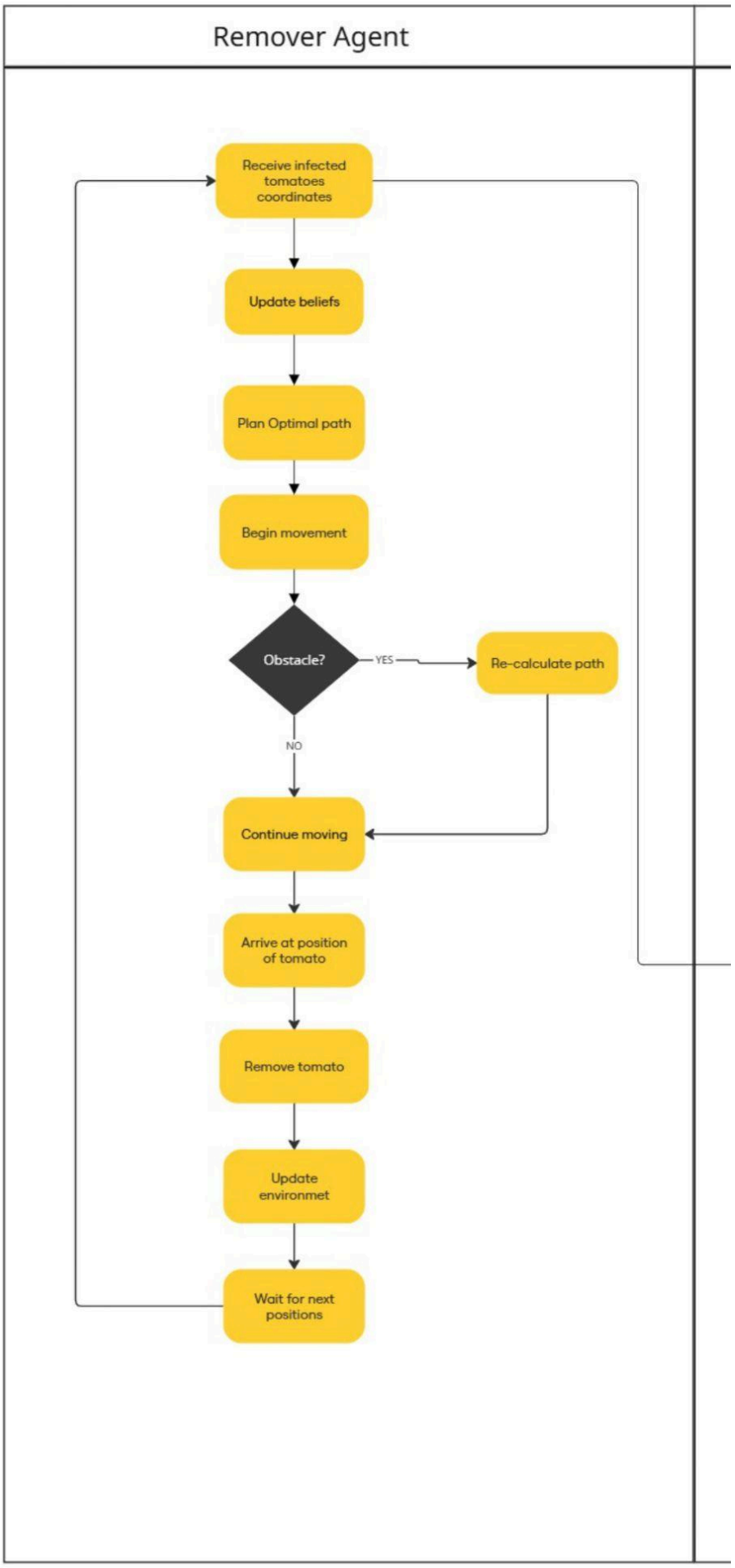
The RemoverAgent maintains a queue of coordinates received from the Analyzer, as well as modules for route planning and obstacle detection. Its methods include planPath(), moveAlongPath(), avoidObstacles(), and removeTomato(), which describe its navigation and action capabilities. Its relationship with the PathPlan class is represented as a dependency to compute a path without necessarily possessing it as part of its identity. Like the other agents, it is related to GridMap through aggregation, as it uses information from the environment to update its status and plan navigation.

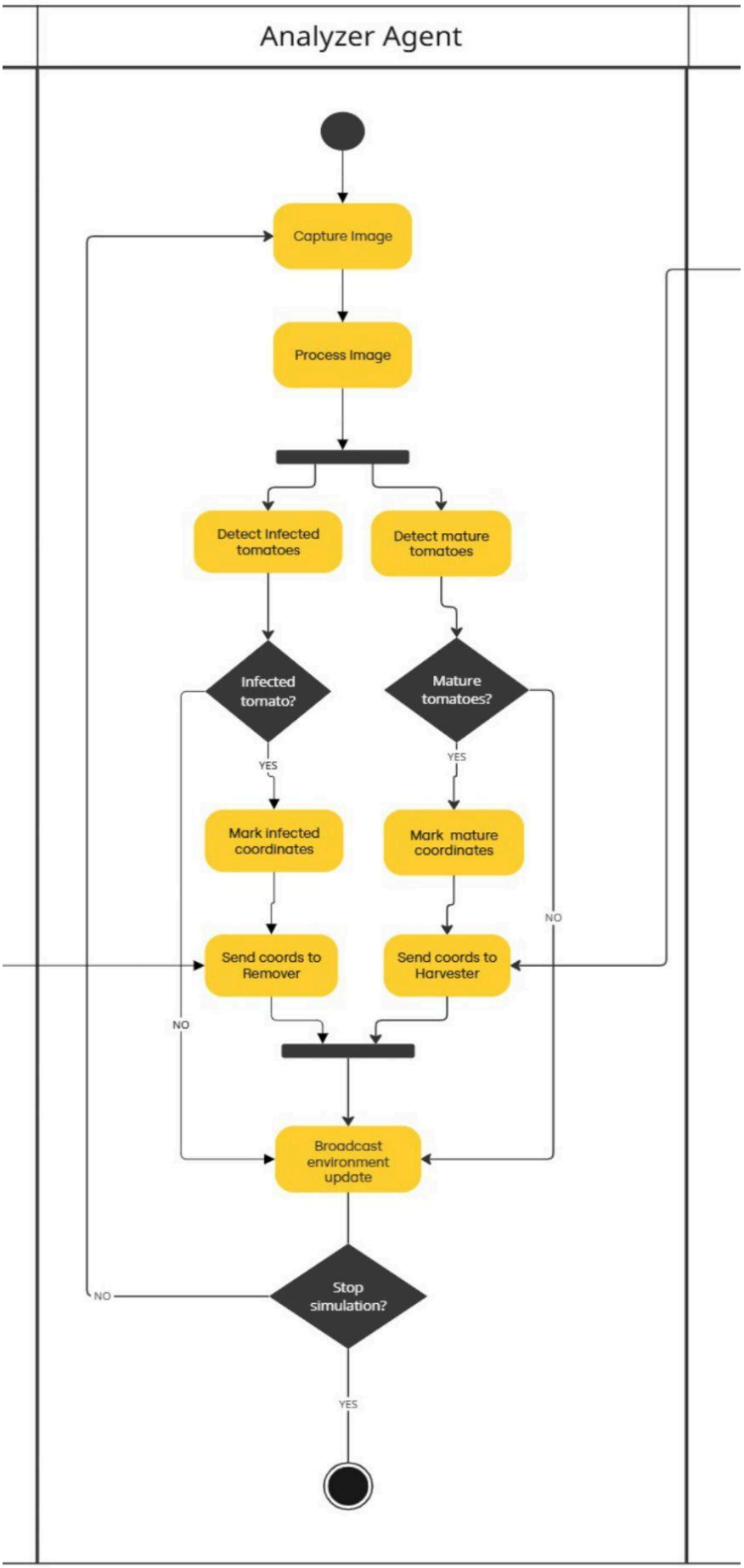
The HarvesterAgent manages a queue of target positions and maintains attributes such as current position, internal state, and access to the environment map. Its main methods include moveToTarget(), harvestTomato(), and updateEnvironment(). Like the RemoverAgent, it works in direct coordination with the AnalyzerAgent, but its functionality is exclusively focused on harvesting.

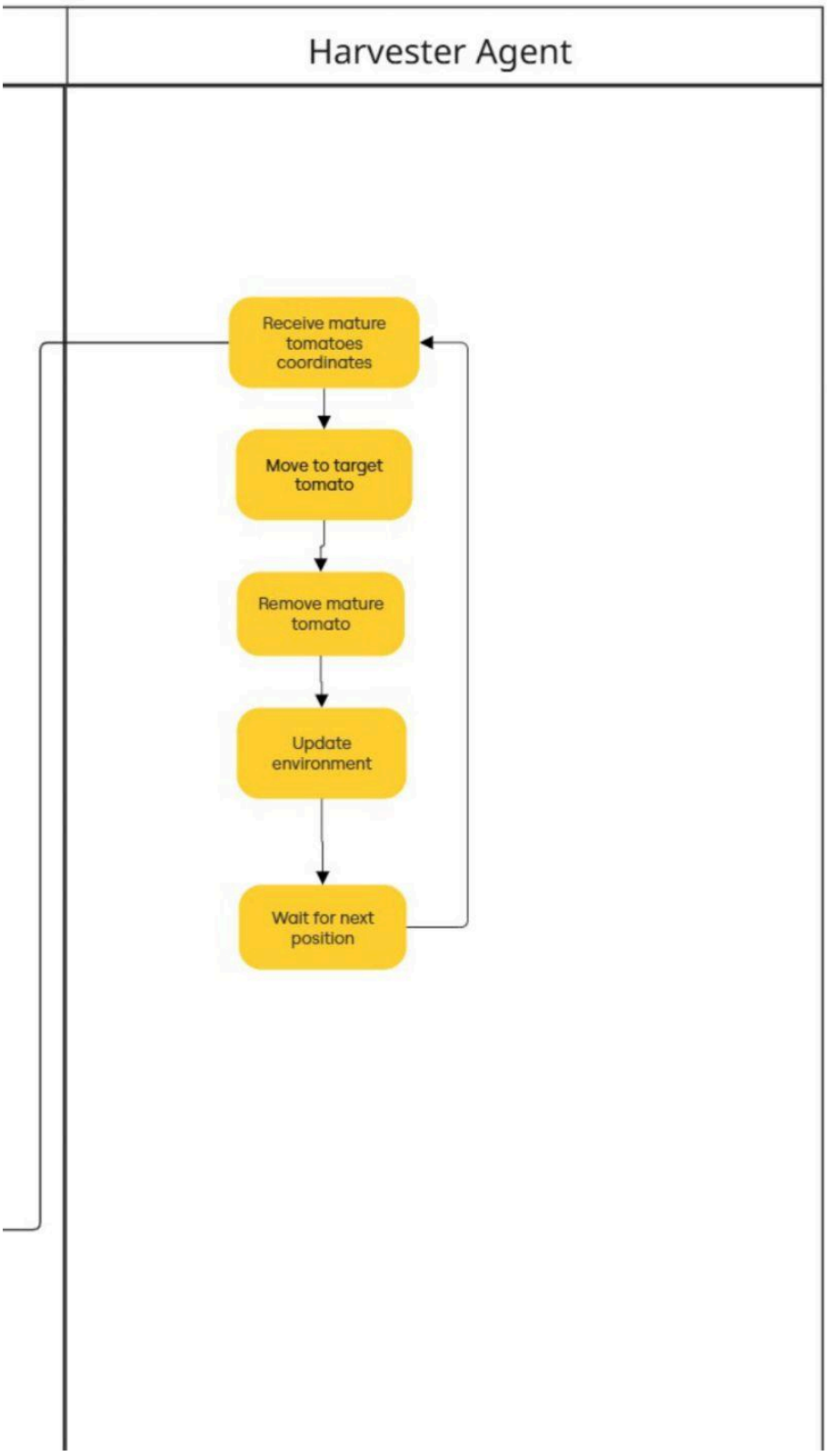
Finally, the auxiliary classes GridMap, DetectionModel, and PathPlan provide support functionalities necessary for agent operation. GridMap models the structure of the environment and offers methods such as isObstacle() and updateCell(). DetectionModel encapsulates a set of weights or parameters to be processed by AI models, and its main method is detect(), used by the Analyzer, and PathPlan defines the logic for calculating routes using computePath.

Activity Diagram




Remover Agent
Receive infected tomatoes coordinates
Update beliefs
Plan Optimal path
Begin movement
Obstacle?
YES
Re-calculate path
NO
Continue moving
Arrive at position of tomato
Remove tomato
Update environmet
Wait for next positions


Analyzer Agent
Capture Image
Process Image
Detect Infected tomatoes
Detect mature tomatoes
Infected tomato?
Mature tomatoes?
YES
YES
Mark infected coordinates
Mark mature coordinates
NO
Send coords to Remover
Send coords to Harvester
NO
Broadcast environment update
Stop simulation?
NO
YES


Harvester Agent
Receive mature tomatoes coordinates
Move to target tomato
Remove mature tomato
Update environment
Wait for next position

The previous diagrams represents the general workflow of the multi-agent system, showing how the Analyzer, Remover, and Harvester agents coordinate their actions throughout the operational cycle. The process begins with the Analyzer. This agent captures images of the environment, processes the information, and classifies the plants according to their status (infected, mature, or unchanged).

Once the analysis results are obtained, the Analyzer evaluates each plant and generates the corresponding information for the executing agents. If infected plants are detected, the Analyzer sends a message to the Remover agent indicating the positions of the tomatoes that must be removed. If mature plants are detected, the Analyzer sends a message to the Harvester Agent to start the harvesting process. If a plant is neither infected nor ripe, the flow simply continues to the next element in the environment without generating any interaction.

After completing the environment analysis or processing a list of plants, the Analyzer evaluates whether the system should continue or whether the simulation has been stopped by the user or the environment. If the system continues, the analysis cycle is repeated; if not, the flow reaches the final state, concluding the overall activity.

Computer Graphics

- **How do you visualize the virtual world?**

We visualize the virtual world as a 3D structured greenhouse environment represented through a grid-based simulation. The world consists of a plantation or crop field where each cell of the grid corresponds to a plant or an empty space. The grid layout allows the agents to navigate, detect conditions, and perform tasks in a controlled environment.

The world is viewed from an overhead perspective, allowing us to display the positions of the agents, plant statuses, and any obstacles. Colors and simple textures are used to differentiate plant states (healthy, mature, infected), and smooth animations indicate movement or task execution by the agents.

- **What virtual elements (buildings, objects, etc.) are key to your proposal?**
  - **Plants:** each plant is a grid cell object with attributes representing its health or maturity status. Plants may visually vary in color
  - **Environmental grid:** a grid or farmland plot that defines navigation zones. Cells may include: walkable areas, obstacles, plantable areas.
  - **Obstacles:** these may include rocks, grass clippings, and small barriers to act as constraints for the Remover and Harvester agent's navigation.
  - **Agents:** Analyzer (drone), Remover robot and Harvester robot. Each is designed using 3D models.

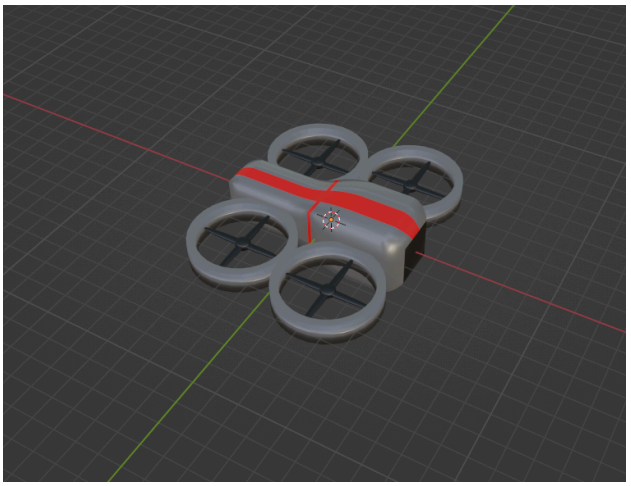
- **Coordinate markers:** small icons showing target positions, queued tasks and visited locations.
- **Field boundaries:** edges, fences, or borders that define the simulation limits.

- **How do you visualize the models of the agents and other relevant objects?**

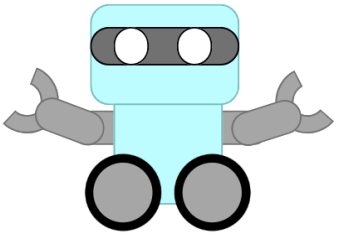
- Analyzer (drone). The Analyzer is visualized as a small quad-copter drone equipped with downward-facing sensors to scan plants. It hovers above the crop grid and moves in straight paths aligned with the environment's cell structure.
- Remover robot. The Remover robot is visualized as a ground robot with wheels or tracks and a mechanical arm used to pull out infected plants.
- Harvest robot. The Harvester robot is visualized as a ground robot with a cutting or collecting tool and a container basket to store harvested tomatoes.
- Tomatoes. Visualized as big red figures, similar to a sphere.
- Obstacles. Obstacles include rocks, crates, or barriers.

- **Include schematics or hand-drawn sketches to support these descriptions.**

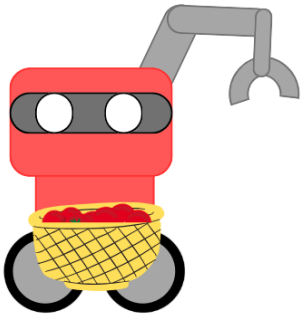
1. **Drone**



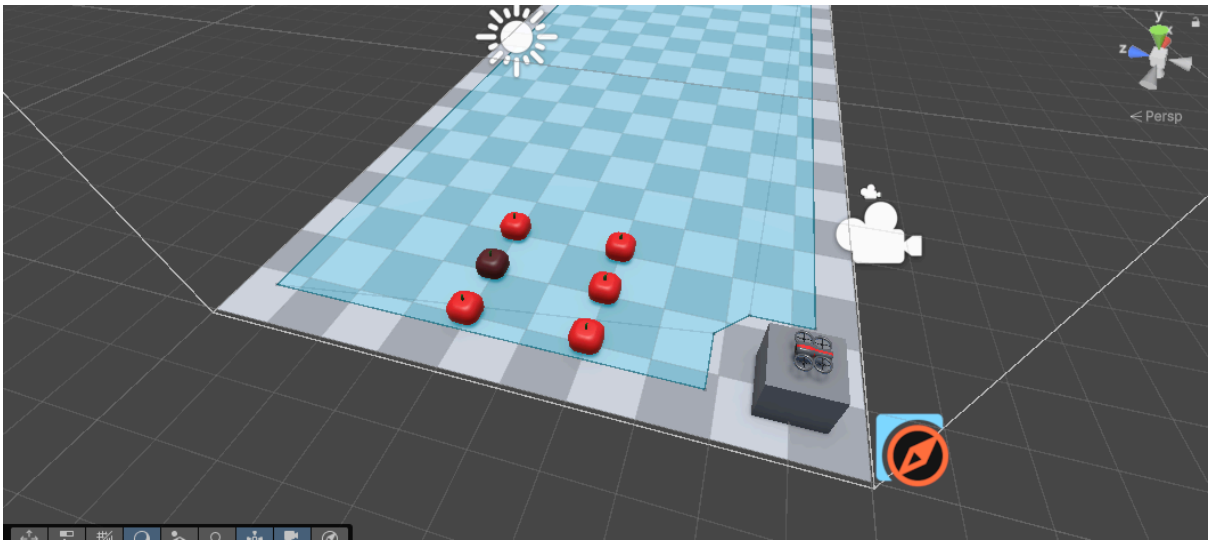
2. **Remover robot**



3. **Harvest robot**



4. **Tomatoes**



5. **Obstacles**

Work Plan

Task	Description	Responsible	Estimated Date	Effort (hours)	Status
Create GitHub repository	**Set up repo for code and documentation**	Jimena	November 10th	0.5	Done

Create communication channel	**(Teams, Discord, etc.)**	Paco	November 10th	0.1	Done
Define simulation environment	**Build initial grid and greenhouse in Unity**	Ilan	November 15th	2.0	Unfinished
Implement Analyzer agent	**Detection and marking system**	Ilan	November 18th	2.0	Unfinished
Implement Removedor agent	**Path planning and removal**	Jimena	November 23th	2.0	Pending
Implement Cosecha agent	**Harvesting logic**	María	November 25th	2.0	Pending
Integrate and test agents	**Validate agents coordination in simulation**	María	November 28th	1.0	Pending
Document progress	**Update proposal and learning**	Paco	November 30th	1.0	Pending

In this deliverable, we learned how to start designing the model and behavior of each of the agents in the system. Throughout the process of creating the Agent Class Diagrams, Standard Class Diagram, Activity Diagram, and the AIP interaction protocols, our team developed a stronger understanding of how to structure the logic of each agent, how to define their roles, and how they communicate with one another. These diagrams allowed us to visualize the system not only as a collection of individual components, but as a coordinated group of autonomous entities that perceive the environment, make decisions, and act accordingly.

One of the key learning outcomes was understanding how to translate abstract requirements into concrete behavioral models. By defining attributes, methods, protocols, and message flows, we were able to identify exactly what information each agent needs, how they process that information, and how they interact with the rest of the system. This helped us clarify responsibilities, for example, distinguishing the perception tasks of the Analyzer from the action-oriented tasks of the Remover and Harvester.

At the same time, this stage of the assignment helped us start defining what the virtual environment will look like. By sketching schematics and developing preliminary visual concepts, we gained insight into how the environment must be organized in order to support autonomous navigation, detection, and task execution. Overall, we learned how to use diagrams not only as documentation tools but as a foundation for building the logic, behavior, and visualization components that will guide the next phase of development.